SHERLOCK

Security Review For Metric



Collaborative Audit Prepared For: **Metric**
Lead Security Expert(s): **eeyore**
TessKimy
Date Audited: **September 22 - September 24, 2026**

Introduction

Metric Update Review focusing on periphery contracts and `MetricPropAmmAdapter.sol`.

Scope

Repository: Metricxyz/metric-periphery

Audited Commit: 378a748793eee3d0b25e8651c0043fc0add1c37c

Final Commit: ed58520348f2eb05ba1903145c0a91f69cd0afde

Files:

- contracts/base/ExternalSwapExecutor.sol
- contracts/base/ExternalSwap.sol
- contracts/base/PeripheryPayments.sol
- contracts/base/SelfPermit.sol
- contracts/base/Sequence.sol
- contracts/libraries/BoundedReturnData.sol
- contracts/MetricOmmSimpleRouter.sol

Repository: Metricxyz/propamm-adapter

Audited Commit: 8f127a2a46d9e90bf23c74a3a08439859423e404

Final Commit: 652d2a1bd0c6181dee42911cdc8cf5ff23d2bd8f

Files:

- contracts/MetricPropAmmAdapter.sol

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	18

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: Anyone can send HyperCore actions as the ExternalSwapExecutor through CoreWriter [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/9

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

On HyperEVM, a contract can act on HyperCore by calling the CoreWriter system contract at `0x3333333333333333333333333333333333333333`. The `ExternalSwapExecutor` calls any `externalRouter` with any calldata, and `externalSwap` lets anyone pick these values. So anyone can make the executor call CoreWriter and send HyperCore actions (for example `spotSend`) from the executor's HyperCore account. Any balance that reaches that account can be taken by the first caller.

Vulnerability Details

`externalSwap` only checks that `externalRouter` is not the router and has code. CoreWriter passes both checks.

The executor then calls the target with the caller's calldata, so the call to CoreWriter comes from the executor and acts on the executor's HyperCore account.

On HyperEVM, a linked ERC20 sent to its system address (`0x20` followed by the token index) is bridged to HyperCore and credited to the sender's HyperCore spot balance. When the external router sends `tokenOut` to the executor, the executor forwards it to `params.recipient`. If the recipient is a system address, the executor is the sender, so the tokens are credited to the executor's HyperCore account and not to the user.

Impact

Executor can be used on HyperCore.

Code snippet

```
if (params.externalRouter == address(this) || params.externalRouter.code.length ==
↪  0) {
  revert InvalidExternalRouter(params.externalRouter);
}
```

```
function swap(IExternalSwap.ExternalSwapParams calldata params) external {
  if (msg.sender != router) revert UnauthorizedCaller();
  IERC20 tokenIn = IERC20(params.tokenIn);

  tokenIn.forceApprove(params.externalRouter, params.amountInMaximum);
  _callExternalRouter(params.externalRouter, params.externalRouterCalldata);
  tokenIn.forceApprove(params.externalRouter, 0);

  uint256 inputAfter = tokenIn.balanceOf(address(this));
  if (inputAfter > 0) tokenIn.safeTransfer(msg.sender, inputAfter);

  IERC20 tokenOut = IERC20(params.tokenOut);
  uint256 outputBalance = tokenOut.balanceOf(address(this));
  if (outputBalance > 0) tokenOut.safeTransfer(params.recipient, outputBalance);
}
```

```
function _callExternalRouter(address target, bytes calldata data) private {
  if (LowLevelCall.callNoReturn(target, data)) return;
  bytes memory reason = BoundedReturnData.read();
  if (reason.length == 0) revert IExternalSwap.ExternalSwapFailed();
  LowLevelCall.bubbleRevert(reason);
}
```

Tools Used

Manual review

Recommendation

Block CoreWriter as an `externalRouter`, and reject HyperCore system addresses as `recipient`:

```
address internal constant CORE_WRITER = 0x3333333333333333333333333333333333333333;
address internal constant HYPE_SYSTEM_ADDRESS =
↪  0x2222222222222222222222222222222222222222;

if (params.externalRouter == CORE_WRITER) revert
↪  InvalidExternalRouter(params.externalRouter);
if (params.recipient == HYPE_SYSTEM_ADDRESS || bytes1(bytes20(params.recipient)) ==
↪  0x20) {
  revert InvalidExternalSwapRecipient(params.recipient);
}
```

Discussion

0xklapouchy

Acknowledged, protocol comment:

> The executor holds no balance between transactions, on HyperEVM or on HyperCore. The only way value reaches the executor's HyperCore account is a user setting `recipient` to a HyperCore system address, which bridges the output to the sender (the executor) instead of to the user. That is a misuse of `recipient`, the same class as sending output to any other wrong address; the user's mistake, not something the router can protect in general (the same happens with `exactInput*` where the pool is the sender). Blocking CoreWriter would only change the outcome of that mistake from ”claimable by anyone” to ”frozen forever”; neither returns funds to the user, and we prefer not to add chain-specific constants to a chain-agnostic router.
> Note on the proposed recipient check: `bytes1(bytes20(recipient)) == 0x20` rejects every address whose first byte is `0x20` (1 in 256 ordinary addresses), not just system addresses.

Issue L-2: Executor should be bound to the router, not passed by the caller [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/10

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`externalSwap` takes `executor` as a caller parameter and sends up to `amountInMaximum` of the caller's tokens to it. The router does not check that this is the official `ExternalSwapExecutor`. This makes the attack surface larger than needed. For example, a compromised frontend or a malicious integrator can set `executor` to an attacker contract, `amountInMaximum` to the user's full balance and `amountOutMinimum` to zero, and the user's router approval is emptied in one call.

Code snippet

```
function externalSwap(address executor, bool payerIsRouter, bool refundAsNative,
↪  ExternalSwapParams calldata params)
  external
  payable
  override
  nonReentrant
  returns (uint256 amountOut, uint256 amountSpent)
```

```
address payer = payerIsRouter ? address(this) : msg.sender;

pay(params.tokenIn, payer, executor, params.amountInMaximum);
uint256 refundBalanceBefore = IERC20(params.tokenIn).balanceOf(address(this));
uint256 recipientBalanceBefore =
↪  IERC20(params.tokenOut).balanceOf(params.recipient);

ExternalSwapExecutor(executor).swap(params);
```

Tools Used

Manual review

Recommendation

Deploy the executor in the router constructor and store it as an immutable. Remove the `executor` parameter:

```
ExternalSwapExecutor public immutable EXECUTOR;

constructor(address weth, address factory) MetricOmmSwapRouterBase(factory)
↪  PeripheryPayments(weth) {
  EXECUTOR = new ExternalSwapExecutor(address(this));
}
```

Discussion

0xklapouchy

Acknowledged, protocol comment:

> Binding the executor does not reduce what a malicious frontend or integrator can do. With the official executor, the same attacker can set `externalRouter` to their own contract and `amountOutMinimum = 0`, and receive the executor's approval of `amountInMaximum`; or simply build an `exactInput` with themselves as recipient. A user who signs router calldata already trusts whoever built it. Keeping `executor` as a parameter lets us deploy a new or restricted executor (e.g. the target allow-list suggested in #13) without redeploying the router.

Issue L-3: ExternalSwap and sequence emit no events [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/11

Summary

`externalSwap` and `sequence` do not emit any events. With the fallback flow, the only way to know which path ran (Metric pool or external router) or why a step failed is to trace the transaction. This makes it hard to monitor the fallback rate, find broken routes and investigate incidents. For example, if the Metric step fails in most blocks because the price push is late, nobody will see it without replaying transactions.

Code snippet

```
if (amountOut < params.amountOutMinimum) revert
↪  ExternalSwapInsufficientOutput(amountOut, params.amountOutMinimum);
if (payerIsRouter) return (amountOut, amountSpent);
_refundInput(params.tokenIn, unspend, refundAsNative);
```

```
for (uint256 i = 0; i < calls.length; i++) {
  SequenceCall calldata step = calls[i];
  (bool success, bytes memory result) = _executeSequenceStep(step.data);
  results[i] = result;
  statuses[i] = success ? StepStatus.SUCCESS : StepStatus.FAILURE;

  if (success) {
    if (step.onSuccess == OnStepSuccess.STOP) break;
  } else {
    if (step.onFailure == OnStepFailure.STOP) break;
    if (step.onFailure == OnStepFailure.REVERT) {
      revert SequenceFailed(results, statuses);
    }
  }
}
```

Tools Used

Manual review

Recommendation

Emit an event after each external swap and after each failed step:

```
event ExternalSwapExecuted(
  address indexed sender,
  address indexed externalRouter,
  address tokenIn,
  address tokenOut,
  uint256 amountSpent,
  uint256 amountOut,
  address recipient
);

event SequenceStepFailed(address indexed sender, uint256 index, bytes4 reason);
```

Discussion

0xklapouchy

Fixed in PR#62

Issue L-4: Sequence cuts step results at 4096 bytes without telling the caller [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/12

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`sequence` copies at most 4096 bytes of each step's return or revert data. When a result is longer, it is cut without any flag, so the caller gets broken ABI data and cannot tell that it was cut. For example, a step that calls `multicall` with five swaps that each return 1000 bytes returns about 5 KB. The caller gets only the first 4096 bytes, and `abi.decode` fails offchain. The same happens to long revert reasons inside `SequenceFailed`.

Code snippet

```
function _executeSequenceStep(bytes calldata data) private returns (bool success,
↪  bytes memory result) {
  success = LowLevelCall.delegatecallNoReturn(address(this), data);
  result = BoundedReturnData.read();
}
```

```
function read() internal pure returns (bytes memory result) {
  uint256 size = LowLevelCall.returnDataSize();
  if (size > 4096) size = 4096;
  result = new bytes(size);
  assembly ("memory-safe") {
    returndatacopy(add(result, 0x20), 0, size)
  }
}
```

Tools Used

Manual review

Recommendation

Tell the caller when data was cut. For example, return the original size next to the data:

```
function read() internal pure returns (bytes memory result, uint256 fullSize) {
  fullSize = LowLevelCall.returnDataSize();
```

```
  uint256 size = fullSize > 4096 ? 4096 : fullSize;
  result = new bytes(size);
  assembly ("memory-safe") {
    returndatacopy(add(result, 0x20), 0, size)
  }
}
```

Or add a `TRUNCATED` value to `StepStatus`, and document the 4096 byte limit in `ISequence`.

Discussion

0xklapouchy

Acknowledged, protocol comment:

> The cap is deliberate (bounds memory expansion from hostile or large return data) and is already stated in the `ISequence.sequence` NatSpec. A result of exactly 4096 bytes is the truncation signal for off-chain decoders. Returning the full size or a `TRUNCATED` status would change the return ABI for a case that the intended use (one swap per step) does not hit.

Issue L-5: Anyone can leave approvals on `ExternalSwapExecutor` [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/13

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`ExternalSwapExecutor.swap()` calls any `externalRouter` with any calldata for any router caller, and afterwards resets only the `tokenIn` allowance given to `externalRouter`. Every other approval made during the call stays on the shared executor, whether an ERC-20 allowance to any spender or a Permit2 allowance. A later user can therefore lose tokens that sit on the executor while their external route runs.

Vulnerability Detail

`ExternalSwap.externalSwap()` only rejects `externalRouter == address(this)` and addresses without code, so token contracts and Permit2 are accepted targets, and the executor calls them as itself:

1. A caller calls `externalSwap(executor, false, false, params)` with `amountInMaximum = 0`, `amountOutMinimum = 0`, `externalRouter = tokenX` and `externalRouterCalldata = approve(spender, type(uint256).max)`. The only cost is gas.
2. The executor runs `tokenX.approve(spender, max)`, then resets only `tokenIn`'s allowance (`tokenIn.forceApprove(params.externalRouter, 0)`), so `tokenX.allowance(executor, spender) == max` persists.
3. Later, a user swaps `tokenX` through the same executor. The router sends `amountInMaximum` to the executor, and the route pulls only part of it (a partial fill) or pays the output to the executor in several transfers.
4. If the route hands control to code the spender controls (for example a hooked pool or a token with transfer hooks), that code calls `tokenX.transferFrom(executor, ...)` while the executor still holds the user's tokens.
5. The executor forwards 0 to the router (`unspend = 0`, `amountSpent = amountInMaximum`). `amountOutMinimum` checks only the output, so the swap still succeeds.

The router design expects no contract in the system to hold a balance after execution, as with Uniswap or 1inch routers. That holds between transactions, but not in the middle of a user's route, which is exactly when these approvals can be used. User allowances to the router itself are not at risk: the router spends allowances only in `pay()`, where the payer is always `msg.sender` or the router.

The same arbitrary call makes any allowance given to the executor itself public: anyone can call `externalSwap` with `externalRouter = token` and `externalRouterCalldata = transferFrom(user, attacker, amount)`. Only the contract NatSpec warns about this ("Users approve the router, never this contract."), not the user-facing documentation.

Because `externalSwap` can be combined in `multicall` and `sequence` flows, a standard user depends on the frontend to build a route that leaves nothing on the executor. Sophisticated callers control their own routes.

Impact

A user can lose the unused part of `amountInMaximum` and any output above `amountOutMinimum` if their external route gives control to whoever set the approval while the executor holds their tokens.

Code Snippet

https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/blob/ddf017bd50339908feba3dbdcb138632c2bc5c88/metric-periphery/contracts/base/ExternalSwapExecutor.sol#L30

Tool Used

Manual Review

Recommendation

Keep the router permissionless and move the restriction into the executor, which the protocol deploys and the frontend selects:

- Deploy protocol-owned executors that only call a fixed set of supported targets (known aggregator routers), either hardcoded at deployment or managed by an owner role on the executor, not on the router.
- Have the frontend always pass one of these trusted executors.
- State clearly in the user and integrator documentation, not only in the NatSpec, that the executor must never receive an approval, because anyone can spend it.

Discussion

0xklapouchy

Acknowledged, protocol comment:

Approving the executor is an integrator error: it is an arbitrary-call contract and its NatSpec already states "Users approve the router, never this contract". The residual-approval scenario additionally requires a victim route that leaves the victim's tokens on the executor during the external call (input not yet pulled, or output the calldata pays to the executor) *and* hands control to attacker code at that moment, i.e. the route itself goes through an attacker-controlled contract, which could already harm the victim up to `amountOutMinimum`. Re-entering the executor is not possible: `swap` is callable only by the router, whose entry points share one lock. A target allow-list would close the scenario but needs an owner role, which we do not want on the router or executor.

Issue L-6: Out-of-gas Metric step starves the `sequence` fallback [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/14

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `sequence` fallback was added so that wallets (MetaMask, Rabby) can simulate Metric swaps: the one-block Metric price is expired in the wallet's simulated block, so the Metric step fails and an external-router step succeeds. The wallet therefore sizes the gas limit, and shows the balance-change preview, for the **fallback** path. On-chain, when the builder's price is present, the **Metric** path runs instead. If it costs more gas than the estimate, it runs out of gas, `sequence` treats that like any other step failure, and the fallback is left with about 1/64 of the gas. The transaction then reverts in exactly the case the update was built for, or, with a STOP/CONTINUE policy, succeeds while leaving the caller's ETH on the router.

Vulnerability Detail

`_executeSequenceStep` forwards all remaining gas to each step and reports only a boolean:

```
success = LowLevelCall.delegatecallNoReturn(address(this), data);
result = BoundedReturnData.read();
```

1. **The gas estimate covers the fallback path.** The wallet estimates on a state where the Metric step reverts early and the external swap runs. The limit is roughly `cost(Metric step fails) + cost(external swap)`.
2. **On-chain the Metric path runs.** With the builder's price in the block, step 1 performs the real Metric swap. A multi-hop route, many bins or pool extensions can make it cost more than the estimated limit.
3. **Running out of gas is an ordinary failure.** The step returns `success = false` with empty return data, exactly like a price revert, so `onFailure = CONTINUE` moves to the fallback. By the 63/64 rule, the fallback receives about 1/64 of the original gas and fails too, and with `REVERT` the whole transaction reverts. The user pays gas for a swap that would have succeeded.
4. **A non-REVERT policy commits partial state.** The `sequence` frame keeps enough gas to return normally. If the failing step's policy is `STOP` (or `CONTINUE` on the last step), the transaction succeeds while that step's cleanup (`refundETH`, `sweepToken`) is undone with it, and the caller's `msg.value` stays on the router for the next caller. This needs no wrong parameters, only a gas limit slightly below the path that actually runs.

The same simulation drives the wallet's balance-change preview (Rabby), so the user is shown the external router's price rather than the Metric price that usually executes. Users see a worse quote than they will get, and some may abandon the swap.

Impact

A Metric swap that needs more gas than the wallet estimated for the fallback path reverts, and the user loses the gas.

Code Snippet

https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/blob/ddf017bd50339908feba3dbdcb138632c2bc5c88/metric-periphery/contracts/base/Sequence.sol#L35-L42

Tool Used

Manual Review

Recommendation

- **In `Sequence`:** add an optional `gasLimit` to `SequenceCall`, where zero keeps today's behaviour. Before a step with a limit runs, revert unless `gasleft()` covers the limits of this step and every later step, each scaled by 64/63 for the 63/64 rule, plus a small overhead per step. Then forward exactly the step's limit. `eth_estimateGas` looks for the lowest gas limit that does not revert, so the wallet's estimate then covers every step's budget even though its simulated Metric step fails early. A Metric step that fails late, or runs out of its own budget, also leaves the fallback its full budget. Safe's `execTransaction` makes the same kind of check for `safeTxGas`.
- **Without a contract change:** have the frontend set the transaction's gas limit instead of relying on the wallet's estimate. The limit must cover the Metric step and the fallback together, plus a margin, because the Metric step can fail late (for example on `amountOutMinimum`) after spending most of its gas. Unused gas is not charged. This only helps with wallets that keep a gas limit supplied by the dapp.

Discussion

0xklapouchy

Acknowledged, protocol comment:

> Gas-limit selection is an off-chain/frontend responsibility: the frontend sets a limit covering the Metric path and the fallback plus margin, which removes the issue without changing `SequenceCall`. The partial-state variant (STOP/CONTINUE as the last

policy) is a sequence-construction error; the documented pattern (#16) ends with REVERT.

Issue L-7: Naming, error and NatSpec inconsistencies [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/15

Summary

The new `ExternalSwap` code has several naming and consistency issues. None of them changes behaviour, but they make the code and the generated documentation harder to read and to integrate with.

Vulnerability Detail

1. **`unspend` should be `unspent`.** The variable in `ExternalSwap.externalSwap` holds the input that was not spent. `unspent` is the standard adjective ("unspent funds"); `unspend` reads as a verb meaning "to reverse spending". The interface parameter `returnUnspendAsNative` repeats it.
2. **The interface and the implementation name the same parameter differently.** `IExternalSwap.externalSwap` declares `bool returnUnspendAsNative` and documents it with `@param returnUnspendAsNative`, while `ExternalSwap.externalSwap` names it `refundAsNative`. The ABI is unaffected, but generated documentation and SDKs show two names for one flag.
3. **One condition, two errors.** The router's swaps check the deadline with `_checkDeadline`, which reverts with `IMetricOmmSimpleRouter.TransactionExpired(deadline, timestamp)`. `externalSwap` re-implements the same check and reverts with a new `IExternalSwap.DeadlineExpired(deadline, timestamp)`. An integrator who decodes errors must handle both for an expired deadline, and a `sequence` whose routes expire returns both (see `test_sequence_preservesBothExpiredSwapReasons`).
4. **The new interfaces are documented less than the existing ones.** `IExternalSwap` and `ISequence` have little NatSpec on errors, structs and parameters compared with `IMetricOmmSimpleRouter` and `IPeripheryPayments`.

Impact

No security impact.

Code Snippet

https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/blob/ddf017bd50339908feba3dbdcb138632c2bc5c88/metric-periphery/contracts/base/ExternalSwap.sol#L16-L24

https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/blob/ddf017bd50339908feba3dbdcb138632c2bc5c88/metric-periphery/contracts/base/ExternalSwap.sol#L42-L43

https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/blob/ddf017bd50339908feba3dbdcb138632c2bc5c88/metric-periphery/contracts/interfaces/IExternalSwap.sol#L28-L45

https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/blob/ddf017bd50339908feba3dbdcb138632c2bc5c88/metric-periphery/contracts/base/MetricOmmSwapRouterBase.sol#L91-L94

Tool Used

Manual Review

Recommendation

- Rename `unspend` to `unspent`, and use one parameter name in both the interface and the implementation, for example `refundAsNative`.
- Use `_checkDeadline` in `externalSwap`, so an expired deadline always reverts with `TransactionExpired`, and remove `DeadlineExpired`.
- Bring the NatSpec of `IExternalSwap` and `ISequence` to the level of the other router interfaces.

Discussion

0xklapouchy

Fixed in PR#62

Issue L-8: Router integration pitfalls are not documented [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/16

Summary

The router should hold nothing after a transaction, and the caller must add `refundETH`, `sweepToken` or `unwrapWETH9` calls to collect anything left. This update adds `externalSwap` and `sequence`, which make batches longer and give each step its own success and failure policy. Several ordinary-looking batches now end successfully while funds stay on the router, where any caller can take them. These are caller or frontend mistakes, not protocol bugs. But standard users cannot see them in the wallet, and neither the README nor the NatSpec lists them.

Vulnerability Detail

In each case the leftover is open to anyone: `sweepToken`, `unwrapWETH9` and `refundETH` pay any caller, and `pay()` spends the router's whole ETH balance for the next WETH payer.

ETH and tokens left on the router

#	Pitfall
1	`externalSwap` with `msg.value > amountInMaximum`, or ETH sent while `tokenIn != WETH` or `payerIsR`
2	`refundAsNative = true` read as "return all my ETH"
3	Router-funded `externalSwap` (`payerIsRouter = true`)

`sequence` policies

#	Pitfall	What happens
4	The last step is set to `onFailure = STOP` or `CONTINUE`	If every route fails, `se`
5	Cleanup (`refundETH`, `sweepToken`) is placed in its own step after the swap	STOP ends the loop a
6	The primary step uses `onSuccess = CONTINUE`	The fallback also runs

#	Pitfall	What happens
7	Different limits on each route	When the fallback ru
8	The gas limit comes from the wallet's estimate	The wallet measures t

Other

#	Pitfall	What happens
9	Approving the `ExternalSwapExecutor`	Anyone can spend that allowance through `externalSwa`
10	Decoding `sequence` results	Each result, and the executor's bubbled revert reason, is

Design behaviour and integration assumptions to document

None of these is a defect by itself. Each one is behaviour that an integrator can rely on wrongly if it is not written down.

#	Behaviour
11	`ExternalSwapExecutor` forwards its **whole** `tokenIn` and `tokenOut` balance to the router and the re
12	The executor approves and calls the same address (`externalRouter`)
13	The executor is ERC-20 only: `swap` is not payable and the executor has no `receive`
14	The router does not check the `executor` argument
15	The external fallback in `sequence` is not simulation-only
16	Wallets simulate on the previous block, where the one-block Metric price has expired
17	`sequence` treats running out of gas exactly like a revert, and returns success when no step revert
18	The new reentrancy lock is taken only by the router's own entry points

Canonical safe fallback

```
sequence{value: X}([
  { multicall([exactInputSingle(..., amountIn: X, min: M), refundETH()]),
  ↪                onSuccess: STOP, onFailure: CONTINUE },
```

```
  { multicall([externalSwap(trustedExecutor, false, true, {amountInMaximum: X, min:
  ↪  M}), refundETH()]), onSuccess: STOP, onFailure: REVERT   },
])
```

Each route is one atomic `multicall` that includes its own cleanup, both routes use the same limits, and the sequence ends with `REVERT`.

Impact

A user whose frontend builds one of these batches loses the leftover ETH or tokens to the next caller of `refundETH` or `sweepToken`, with no warning from the wallet simulation.

Code Snippet

https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/blob/ddf017bd50339908feba3dbdcb138632c2bc5c88/metric-periphery/contracts/base/Sequence.sol#L29-L43

Tool Used

Manual Review

Recommendation

- Add both lists and the canonical safe fallback to the integrator documentation, and the relevant items to the `ISequence`, `IExternalSwap` and `ExternalSwapExecutor` NatSpec.
- Optionally, remove the most common pitfalls in code without adding any owner role:
  - At the end of `sequence`, send any ETH left on the router to `msg.sender`. This covers pitfalls 1, 4 and 5.
  - In `externalSwap`, refund any `msg.value` that `pay()` did not wrap. This covers pitfall 1.

Discussion

0xklapouchy

Fixed in PR#62 and the router docs

Issue L-9: NatSpec inaccuracies and minor hardening [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/17

Summary

Several comments in `MetricPropAmmAdapter` no longer match the code, and two code paths can be tightened. None of them changes the adapter's behaviour with honest pools.

Vulnerability Detail

1. **Tie-break comment.** `_selectBest()` says "Registration order, so the first-registered candidate keeps winning ties." `_removePool()` moves the last pool into the freed slot. After any removal, a later-registered pool can come first and win ties.
2. **`Swapped.amountOut`.** IPropAMM documents it as "measured as a balance delta", but the adapter emits the pool's settled delta. The two match for standard ERC-20s and differ for tokens with transfer fees, which the README excludes.
3. **Callback shape check.** In swap mode, `metricOmmSwapCallback()` pays every positive delta in either token and never checks that only the input token is owed. The periphery router's callback rejects `amount0Delta <= 0 && amount1Delta <= 0` and pays only the single positive amount, in the input token. With factory-verified pools, a wrong shape reverts later in `_execute()`, so the check is defensive only.
4. **Generation probe.** The eight-argument branch of `_assertSupportedGeneration()` admits only pools of Metric's previous factory, `0xA32761549A1DE40060c194C86A8df24f0A29bA2d`. There are 92 of those pools, on Ethereum, BNB, Base, Arbitrum and Robinhood. They are abandoned: most are empty, and none of the pools with balances on both sides returned a quote at the current block. Every Metric pool that traded in the last three days is of the nine-argument generation. Pools of the oldest generation return 13 words from `getImmutables()` instead of 24, and `addPool()` reverts on them without a reason instead of with `UnsupportedPoolGeneration`.

Impact

No security impact.

Code Snippet

https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L391

https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L230-L246
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/interfaces/IPropAMM.sol#L11-L28
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L706-L717
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L187-L228

Tool Used

Manual Review

Recommendation

- Fix the tie-break comment. If registration order should decide ties, keep the list order on removal (shift the entries instead of swap-and-pop).
- Document that `Swapped.amountOut` is the pool's settled delta, or measure the recipient's balance change instead.
- In swap mode, require exactly one positive delta and require it to be `tokenIn`, as the periphery router does.
- Drop the eight-argument branch unless Metric plans to revive those pools. Decode `getImmutables()` defensively, so that every unsupported pool reverts with `UnsupportedPoolGeneration`.

Discussion

semyon-metric

Confirmed. Fixed in Metricxyz/propamm-adapter PR #3 (a private repository, so the relevant changes are quoted here).

1. **Tie-break comment** (commit `4947fea`). It now reads: "List order decides ties, the earlier candidate winning. That is registration order only until a removal: `_removePool` moves the last candidate into the freed slot. Tied candidates fill the same amount, so the order costs the trader nothing." We kept swap-and-pop, since tied candidates fill the same amount.
2. **`Swapped.amountOut`** (commit `4947fea`). The `swap()` NatSpec now says `Swapped` reports `amountIn` as pushed by the caller and `amountOut` as the pool settled it. For the supported standard ERC-20s that equals the recipient's balance change.
3. **Callback shape** (commit `8817fd6`). `_execute` passes the direction as callback data, and in swap mode the callback pays the input token only:

```
// _execute
IMetricOmmPoolActions(pool).swap{gas: LIVE_ATTEMPT_GAS_LIMIT}(recipient,
↪  zeroForOne, ..., abi.encode(zeroForOne), "");
// metricOmmSwapCallback, swap mode
(int256 owed, address tokenIn) =
  abi.decode(data, (bool)) ? (amount0Delta, info.token0) : (amount1Delta,
  ↪  info.token1);
if (owed > 0) IERC20(tokenIn).safeTransfer(pool, uint256(owed));
```

The output token is never paid, whatever its delta. Any other shape still reverts in `_execute`. Test: `test_callback_paysOnlyTheInputToken`, in both directions.

4. **Generation probe** (commit `86c06dd`). The eight-argument branch is removed, and `_assertSupportedGeneration` probes only the nine-argument `simulateSwapAndRevert`. We did not add the defensive `getImmutables()` decode. A pool of the oldest generation still reverts, only without a reason, and the pool manager lists only the configured factory's pools.

0xklapouchy

Fixed in PR#4

Issue L-10: NatSpec is missing for events, errors and parameters [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/18

Summary

Most functions of `MetricPropAmmAdapter` have detailed NatSpec, but its events, custom errors and several parameters have none. One statement about the quote recipient is also inaccurate.

Vulnerability Detail

1. None of the five events has NatSpec: `PoolAdded`, `PoolRemoved`, `MaxQuoteDeviationUpdated`, `FactoryAllowed` and `MaxPoolsPerPairUpdated`.
2. 18 of the 19 custom errors have no NatSpec; only `QuoteResult` has one.
3. External functions have a `@notice` but no `@param`.
4. The `_quotePool` NatSpec says that `recipient` is forwarded "so recipient-aware extensions see the same context the live swap would give them". This is true for the selection inside `swap()`. It is not true for `quote()`, which passes the adapter itself as recipient.

Impact

No security impact.

Code Snippet

https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L98-L123
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L465-L469
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L595-L596

Tool Used

Manual Review

Recommendation

- Add NatSpec to the events, the errors, `BPS` and `PoolInfo`.
- Add `@param` tags to the functions.
- State in the `_quotePool` NatSpec that `quote()` uses the adapter as recipient, so recipient-aware extensions can see a different context in `quote()` than in `swap()`.

Discussion

semyon-metric

Confirmed. Fixed in commit `4947fea` of Metricxyz/propamm-adapter PR #3 (a private repository, so the relevant changes are quoted here).

- NatSpec is added to all five events, every custom error, `PoolInfo` and `BPS`.
- `@param` and `@return` tags are added to the external functions that do not inherit them from `IPropAMM`.
- The `_selectBest` and `_quotePool` NatSpec now says that `quote()` forwards the adapter itself as recipient. A recipient-aware extension can therefore answer `quote()` differently from `swap()`.

0xklapouchy

Fixed in PR#8

Issue L-11: `sweep()` gives almost no protection to stray tokens [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/19

Summary

The adapter uses the router's push-payment model: tokens are safe only when they arrive in the same call as `swap()`. A balance left on the adapter between calls is a known caller mistake, the same trade-off routers accept for tokens left on them. The adapter offers `sweep()` to recover such tokens, but it gives almost no protection. `swap()` is permissionless and pays the pool from the adapter's balance, so anyone can take a stray balance before the admin sweeps it.

Vulnerability Detail

`swap()` expects `amountIn` to "already sit on this contract", and the callback pays the pool from whatever the adapter holds:

```
if (amount0Delta > 0) IERC20(info.token0).safeTransfer(pool, uint256(amount0Delta));
if (amount1Delta > 0) IERC20(info.token1).safeTransfer(pool, uint256(amount1Delta));
```

This is safe in the router's one-call flow. For tokens that end up on the adapter anyway, the `sweep()` NatSpec promises recovery ("Recover tokens stranded on the adapter"), but:

- Anyone can call `swap(strandedToken, otherToken, balance, 0, attacker, deadline)` without sending anything, and the callback pays the pool with the stranded tokens. A stray balance is free to take, so a bot usually gets it before the admin's `sweep()` lands.
- `swap()` accepts `recipient == address(this)`, directly or through the router, so a mistaken recipient parks the output on the adapter.
- `sweep()` can reliably recover only tokens that no registered pair uses, because `swap()` reverts with `UnsupportedPair` for them.

Impact

`sweep()` protects almost none of the tokens it is meant for.

Code Snippet

https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L255-L258

https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L712-L716
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L618-L626

Tool Used

Manual Review

Recommendation

- Restrict `swap()` to the router, so a stray balance stays on the adapter until `sweep()` recovers it. The router always pushes and swaps in one call.
- In `swap()`, revert when `recipient` is the adapter or `address(0)`.
- If `swap()` stays permissionless, state in the `sweep()` NatSpec and the integrator docs that `sweep()` recovers only tokens of unregistered pairs, and that any other balance left on the adapter is lost.

Discussion

semyon-metric

Confirmed, partially adopted, in commit `06def5d` of Metricxyz/propamm-adapter PR #3 (a private repository, so the relevant changes are quoted here).

- `swap()` now rejects the adapter and the zero address as recipient:

```
if (recipient == address(this) || recipient == address(0)) revert
↪  InvalidRecipient(recipient);
```

- The `sweep()` NatSpec, the README and the design spec now say `sweep()` reliably recovers only tokens that no registered pair serves. Input is safe on the adapter only within the call that pushes it and swaps.
- `swap()` stays permissionless. Restricting it to the router would tie the adapter to one router deployment. A stray balance only arises from a caller's mistake, which the push model cannot guard against anyway.

0xklapouchy

Fixed in PR#7

Issue L-12: Runner-up and lone-pool swaps run with no gas cap [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/20

Summary

`swap()` caps the first live attempt at `LIVE_ATTEMPT_GAS_LIMIT`, but runs the runner-up and a lone candidate through `_execute()` with all remaining gas. A registered pool that burns gas on that last attempt starves PropAMMRouter's Uniswap V3 fallback. The trade reverts, or burns most of the trader's gas limit.

Vulnerability Detail

The NatSpec caps the first attempt "so that a winner burning gas cannot leave the runner-up nothing to run on", but the last attempt has no cap:

```
if (!filled) {
  TransientSlot.tstore(TransientSlot.asAddress(EXPECTED_POOL_SLOT), pool);
  amountOut = _execute(pool, tokenIn, amountIn, quoted, recipient);
}
```

This path serves the runner-up, a lone pool, and any pair where only one candidate quotes; on Ethereum that is the usual case. Quotes run under 1M gas and the first attempt under 1.3M. A pool's extension can therefore behave normally in both, burn gas only when `gasleft()` is higher, and keep its quotes healthy.

The router calls the adapter inside `try this._dispatchVenue(...)` without a gas cap, and runs the fallback in the `catch` on the gas the calling frames kept back. Honest pools need far less: a live swap on the Ethereum pools, including `afterSwap`, costs 188k–481k gas.

Impact

A registered pool can make router trades on its pair revert, or burn most of the trader's gas limit, while its quotes look healthy.

Code Snippet

https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L643-L658
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L680-L694
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3

a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L77-L88
https://github.com/lambdaclass/propamm-router-contracts/blob/226af1245bb9e8d056eaa0e1051cf69ba86b7a67/src/PropAMMRouter.sol#L439-L469

Tool Used

Manual Review

Recommendation

Run the last attempt through `this.executeCandidate{gas: LIVE_ATTEMPT_GAS_LIMIT}(...)` as well, for the runner-up and for a lone candidate. If callers should see the runner-up's revert reason, bubble up its revert data after the call.

Discussion

semyon-metric

Confirmed, fixed in commit `5d19089` of Metricxyz/propamm-adapter PR #3 (a private repository, so the relevant changes are quoted here).

Every live attempt now calls the pool with the cap, the runner-up's and a lone candidate's included:

```
(int128 amount0Delta, int128 amount1Delta) = IMetricOmmPoolActions(pool).swap{gas:
↪  LIVE_ATTEMPT_GAS_LIMIT}(...);
```

New tests `test_swap_capsTheGasOfALoneCandidate` and `test_swap_capsTheGasOfTheRunnerUp` use an `afterSwap` that burns all its gas. With a 10M allowance, the failing swap now spends under 2M for a lone candidate and under 4M for a runner-up. Without the fix, both spend about 9.7M.

0xklapouchy

Fixed in PR#5

Issue L-13: Forged `QuoteResult` payloads are accepted as quotes [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/21

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_quotePool()` accepts any revert data shaped like `QuoteResult(int256,int256)` as the pool's quote, and a registered pool's `beforeSwap` extension can revert with such data. PropAMMRouter can then choose Metric with a quote that no pool fills, and two forging pools make every `swap()` on the pair revert. The NatSpec says the payload cannot be authenticated, but it can.

Vulnerability Detail

The `_decodeQuoteResult()` NatSpec accepts the risk: this function ”can only make a forged payload harmless to the OTHER candidates, never detect it: registration is where a pool is trusted.” But pool creation is permissionless, the creator picks the extensions, and a forging pool passes health checks based on quotes.

The forged quote is not harmless:

1. `quote()` returns the best quote within the 1% band. With an honest pool at 999,994 and one forging pool, `quote()` reported 1,009,500 while the swap filled 999,994. The router therefore picks Metric over better venues.
2. In `swap()`, `FillBelowQuote` rejects the forger's fill and the honest runner-up fills instead. A trader whose `amountOutMin` came from the quote gets the Uniswap V3 fallback, or a revert.
3. Two forging pools take both live attempts, so every `swap()` on the pair reverts.

Authentication is possible:

- The pool passes `callbackData` only to the callback, never to extensions.
- The pool runs under a fixed gas cap, so an extension cannot see how much gas the adapter has.

A per-quote value derived from `gasleft()` and a counter, passed as `callbackData` and echoed in `QuoteResult`, therefore cannot be forged. Separately, a genuine quote never exceeds the pool's `tokenOut` balance, which `_selectBest()` already reads.

Impact

A registered pool can make the router choose Metric with a quote that no pool fills, so traders get the Uniswap fallback.

Code Snippet

https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L524-L554
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L506-L519
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L371-L384
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L593-L598

Tool Used

Manual Review

Recommendation

- Pass a per-quote value derived from `gasleft()` and a counter as `callbackData`, keep it in transient storage, and accept only `QuoteResult` payloads that carry it.
- Skip a candidate whose quote exceeds its `tokenOut` balance.

Discussion

semyon-metric

Acknowledged, mitigated differently, in commit `6627da3` of Metricxyz/propamm-adapter PR #3 (a private repository, so the relevant changes are quoted here).

We agree the payload could be authenticated: an extension sees neither the callback data nor the adapter's transient storage. We did not add that, because it does not remove the impact:

- A registered pool whose price provider prices the quote high, and whose `afterSwap` rejects the live swap, wins selection the same way with a genuine payload. A quote never reaches `afterSwap`.
- Two such pools take both live attempts, just as two forgers would.

What forging saves the attacker is the liquidity behind the quote. So we adopted your second recommendation, and selection now skips a quote above the pool's own `tokenOut` balance:

```
(ok, standing) = _standingBalance(tokenOut, list[i]);
if (!ok || out > standing) continue;
```

The `_decodeQuoteResult` NatSpec and the README now say the payload is not authenticated and why, instead of saying it cannot be. Registration remains the trust boundary. Test: `test_consensus_skipsAQuoteAboveThePoolsBalance`.

0xklapouchy

Acknowledged, mitigated in PR#6: a quote above the pool's own `tokenOut` balance is now skipped, and the NatSpec and README document that the payload is not authenticated.

Issue L-14: Consensus band can drop the best honest quote [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/22

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_selectBest()` drops every quote more than `maxQuoteDeviationBps` (1% by default) above or below the consensus. The consensus is the lower median of the quotes, weighted by each pool's raw `tokenOut` balance. When honest pools of a pair price more than 1% apart, the better one is dropped. The raw balance also lets tokens that no trade reaches set the consensus.

Vulnerability Detail

The NatSpec assumes that honest pools "land close together", and that moving the centre requires holding more `tokenOut` than the honest pools. Neither holds:

1. **Honest pools can differ.** Pools of one pair each have their own spread, fees, price provider and depth. Take two pools with equal balances quoting 1,000 and 1,020. The consensus is 1,000, because an even split takes the lower value, and the band is 990–1,010. The 1,020 quote is dropped, and the trader gets 1,000.
2. **Balance is not depth.** `balanceOf(pool)` counts bins far from the price, unpaid fees and plain transfers. A pool with a large balance far from the price quotes worse for large trades, yet still sets the consensus, and that liquidity can be removed after the trade. Such balances exist: older Metric pools on BNB hold more than $1.5k and return zero output at every trade size.

Dropping quotes above the band is not needed in `swap()`, because `FillBelowQuote` already holds the winner to its own quote.

Impact

Traders are filled at a worse price than an honest registered pool offers, by up to the gap between the two pools.

Code Snippet

https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L349-L357
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3

a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L371-L401
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L404-L437
https://github.com/sherlock-scoping/Metricxyz__propamm-adapter/blob/30904f3af3a53c7cb9057ababe30927a31b22839/contracts/MetricPropAmmAdapter.sol#L445-L454

Tool Used

Manual Review

Recommendation

- Weigh candidates by their bin reserves (`binTotals`, read through `extsload`), capped at the balance.
- In `swap()`, keep candidates above the band, and apply the band only to what `quote()` reports.

Discussion

semyon-metric

Confirmed, fixed in commit `4918e95` of Metricxyz/propamm-adapter PR #3 (a private repository, so the relevant changes are quoted here).

We adopted your second recommendation. `_selectBest` takes a `capAbove` flag, which `quote()` sets to `true` and `swap()` to `false`:

```
if (out * BPS < lower || (capAbove && out * BPS > upper)) continue;
```

- `swap()` holds its winner to its own quote (`FillBelowQuote`). An inflated quote costs its pool the first attempt, and the runner-up fills. An honest pool above the band now wins there.
- `quote()` keeps the upper cap, because the router compares that answer against other venues before any fill can check it.
- **Both** keep the lower bound, so a far worse pool is never the fallback.

We did not adopt weighting by `binTotals`. It also counts bins far from the price, so it would not address #26.

Tests:

- `test_consensus_swapServesAnHonestQuoteAboveTheBand`: two honest pools 2% apart. `quote()` returns the lower quote, and `swap()` fills from the better pool.
- `test_consensus_swapHoldsAQuoteAboveTheBandToItsFill`.

semyon-metric

Update: after internal review we reverted this change in commit `b0f8981` of Metricxyz/propamm-adapter PR #3. That repository is private, so the reasoning is here.

Without the upper cap, `swap()` spent its two live attempts on the two highest quotes. Two pools quoting above the band whose live swaps fail can then take both attempts, and a size-dependent stop-loss is enough to make them fail. The in-band pool that `quote()` reported is never tried, although the router chose the adapter on that quote. `swap()` again applies the same band as `quote()`.

Status: Acknowledged, won't fix. The cost you describe remains: an honest pool pricing more than the band above the reserve-weighted centre is not served, so a trade can fill below what that pool offers. The admin can widen the band with `setMaxQuoteDeviationBps` for pairs whose pools spread further. Test: `test_swap_neverSpendsItsAttemptsAboveTheBand`.

0xklapouchy

Acknowledged, documented in PR#11: no code change, the NatSpec and README now explain why `swap()` keeps the same band as `quote()`.

Issue L-15: Admin handover cannot finish in the same block, but the NatSpec says there is no delay [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/24

Summary

The constructor NatSpec says the admin handover works "with no delay, as `Ownable2Step` did". The inherited OpenZeppelin contract still forces the new admin to wait at least one block before accepting.

Vulnerability Details

The adapter passes a delay of `0` to `AccessControlDefaultAdminRules`. When the transfer starts, OpenZeppelin sets the schedule to `block.timestamp + 0`. When the new admin accepts, OpenZeppelin checks `schedule < block.timestamp`. In the same block the two values are equal, so the check fails and `acceptDefaultAdminTransfer` reverts with `AccessControlEnforcedDefaultAdminDelay`. The new admin can only accept in a later block. `Ownable2Step` has no such rule, so the comment is wrong.

Impact

Begin and accept cannot be batched in one transaction or one block, for example by a multisig doing both steps together. The comment misleads anyone who plans the handover based on it.

Code Snippet

```
/// @param admin The single `DEFAULT_ADMIN_ROLE` holder. Handing it over takes two
↪  steps, with no
///        delay, as `Ownable2Step` did.
/// @param manager Granted `MANAGER_ROLE` unless zero.
/// @param factories The initially allowed factories.
constructor(address admin, address manager, address[] memory factories)
↪  AccessControlDefaultAdminRules(0, admin) {
```

```
function _beginDefaultAdminTransfer(address newAdmin) internal virtual {
    uint48 newSchedule = SafeCast.toUint48(block.timestamp) + defaultAdminDelay();
    _setPendingDefaultAdmin(newAdmin, newSchedule);
```

```
    emit DefaultAdminTransferScheduled(newAdmin, newSchedule);
}
```

```
function _acceptDefaultAdminTransfer() internal virtual {
    (address newAdmin, uint48 schedule) = pendingDefaultAdmin();
    if (!_isScheduleSet(schedule) || !_hasSchedulePassed(schedule)) {
        revert AccessControlEnforcedDefaultAdminDelay(schedule);
    }
```

```
function _hasSchedulePassed(uint48 schedule) private view returns (bool) {
    return schedule < block.timestamp;
}
```

Tools Used

Manual Review

Recommendation

Fix the NatSpec (and the README) to say the new admin can accept only from the next block onward. If a same block handover is needed, override `_acceptDefaultAdminTransfer` to also accept `schedule == block.timestamp`.

Discussion

semyon-metric

Confirmed. The documentation is fixed in commit `4947fea` of Metricxyz/propamm-adapter PR #3 (a private repository, so the relevant changes are quoted here).

The constructor NatSpec, the README and the design spec now say the delay is zero but the new admin can accept only from the next block. OpenZeppelin requires the schedule to be strictly in the past, so the two steps cannot be batched. `test_admin_handoverTakesTwoSteps` now pins the same-block `AccessControlEnforcedDefaultAdminDelay` revert. We do not need a same-block handover, so `_acceptDefaultAdminTransfer` is not overridden.

0xklapouchy

Fixed in PR#9

Issue L-16: Tie break does not always go to the first registered pool [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/25

Summary

`_selectBest` says the first registered pool wins ties. `_removePool` uses swap and pop, which changes the list order, so after a removal a later registered pool can win the tie.

Vulnerability Details

Ties are broken by position in `_poolsByPair`. When a pool is removed, the last pool in the list is moved into the empty slot. Say pools A, B and C are registered in that order. After A is removed, the list becomes C, B. If B and C quote the same output, C now wins, even though B was registered first. There is no way to restore the order other than removing and adding pools again.

Impact

On a tie, the swap can go to a different pool than the one the documented rule picks. Tied pools give the same output, so the trader loses nothing. The behavior just does not match the comment.

Code Snippet

```
// Registration order, so the first-registered candidate keeps winning ties.
for (uint256 i = 0; i < count; i++) {
  uint256 out = outs[i];
  if (out * BPS > upper || out * BPS < lower) continue;
  if (out > bestOut) {
    (runnerUp, runnerUpOut) = (bestPool, bestOut);
    (bestPool, bestOut) = (pools[i], out);
  } else if (out > runnerUpOut) {
    (runnerUp, runnerUpOut) = (pools[i], out);
  }
}
```

```
for (uint256 i = 0; i < n; i++) {
  if (list[i] == pool) {
    list[i] = list[n - 1];
    list.pop();
```

```
      break;
    }
  }
```

Tools Used

Manual Review

Recommendation

Remove pools in a way that keeps the order, by shifting the later entries left. The list is capped at `maxPoolsPerPair` (at most 100), so this is cheap. Another option is a manager function to reorder a pair's pools. At the least, fix the comment.

Discussion

semyon-metric

Confirmed, a duplicate of #17 item 1. The comment is fixed in commit `4947fea` of Metricxyz/propamm-adapter PR #3 (a private repository, so the relevant changes are quoted here).

It now says list order decides ties, and that this is registration order only until a removal. We kept swap-and-pop, because tied candidates fill the same amount and the order costs the trader nothing.

0xklapouchy

Fixed in PR#4

Issue L-17: Consensus weight can be inflated with liquidity that does not serve the trade [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/26

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Each pool's weight in the consensus is its raw `tokenOut.balanceOf(pool)`. Anyone can raise that balance by adding liquidity to bins far from the current price. The trade never reaches those bins, but they still count as weight.

Vulnerability Details

`_weightedMedian` picks the consensus using these balances, and any quote more than `maxQuoteDeviationBps` away from the consensus is dropped. An attacker can add liquidity to very low or very high bins of a registered pool that quotes worse. The pool's balance, and so its weight, grows until its quote becomes the consensus. A better pool that quotes more than the band above it is then dropped, and the worse pool fills the trade. The attacker can remove the liquidity after the trade and get the capital back. A plain token transfer to the pool raises the weight in the same way.

Impact

Trades can be sent to a pool that pays less than the best registered pool, limited only by the trader's `minAmountOut`.

Code Snippet

```
uint256 standing;
(ok, standing) = _standingBalance(tokenOut, list[i]);
if (!ok) continue;
pools[count] = list[i];
outs[count] = out;
// Capped so that summing the weights can never overflow, whatever the token
↪  reports.
weights[count] = Math.min(standing, type(uint128).max);
count++;
```

```
bool unweighted = total == 0;
if (unweighted) total = count;
uint256 cumulative;
for (uint256 i = 0; i < count; i++) {
  cumulative += unweighted ? 1 : weights[order[i]];
  if (cumulative * 2 >= total) return values[order[i]];
}
return values[order[count - 1]];
```

Tools Used

Manual Review

Recommendation

Weight pools by liquidity the trade can actually use, such as the bins between the current price and the pool's fill for this `amountIn`, not by raw balance. Another option is to stop dropping quotes that sit above the band in `swap()`, since the fill vs quote check (`FillBelowQuote`) and the runner up fallback already stop a fake high quote there.

Discussion

semyon-metric

Confirmed, fixed together with #22 in commit `4918e95` of Metricxyz/propamm-adapter PR #3 (a private repository, so the relevant changes are quoted here).

- `swap()` no longer excludes quotes above the band. The best quote is never below the weighted median, so it is always tried first.
- A centre moved by liquidity far from the price, or by plain transfers, now only decides which pools may be the fallback in `swap()`.
- In `quote()` it can still under-report a better pool. That costs Metric the routing, but not the trader: the router then picks another venue.

The README documents that the weight is the raw balance and what follows from that.

semyon-metric

Update: the #22 change is reverted (see #22), so this is now Acknowledged, won't fix.

Moving the centre takes more `tokenOut` balance than all the honest candidates of the pair hold together. A centre moved down also lowers what `quote()` reports, so the router compares the adapter against other venues at that lower price rather than routing to it on a better one.

0xklapouchy

Acknowledged, documented in PR#11: no code change, the consensus weight is still the pool's raw `tokenOut` balance.

Issue L-18: `Swapped` event NatSpec says `amountIn` is pulled from `sender`, but it is prefunded [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-metric-update-sept-22nd-2026/issues/27

Summary

The `Swapped` event doc says `amountIn` is "pulled from `sender`". The interface uses a push model: the caller sends `tokenIn` to the venue before calling `swap`, and the adapter never pulls anything.

Vulnerability Details

The `swap` NatSpec in the same interface says `amountIn` must already have been sent to the propAMM. The adapter follows that model: it pays the pool from its own balance and never calls `transferFrom` on the sender. The event doc describes a pull model that is not used, so it contradicts the rest of the interface.

Impact

Integrators and indexers reading the event doc may assume `sender` approved and paid the adapter directly, when the router prefunded it. This is a documentation issue only.

Code Snippet

```
/// @notice Emitted once per successful swap after `tokenOut` is delivered to
 ↪  `recipient`.
/// @param sender The address that invoked the swap entrypoint and supplied
/// `amountIn` of `tokenIn`. Indexed so consumers can fetch a given account's
/// recent swaps.
/// @param tokenIn The token sold.
/// @param tokenOut The token bought.
/// @param amountIn The exact amount of `tokenIn` pulled from `sender`.
```

```
/// @dev Expects `amountIn` of `tokenIn` to have ALREADY been transferred to
/// the propAMM by the caller (push-payment).
```

Tools Used

Manual Review

Recommendation

Change the doc to say `amountIn` is the exact amount of `tokenIn` that the caller prefunded to the propAMM before calling `swap`.

Discussion

semyon-metric

Confirmed. `IPropAMM.sol` is LambdaClass's interface and outside the audited scope. Our copy is corrected in commit `4947fea` of Metricxyz/propamm-adapter PR #3 (a private repository, so the relevant changes are quoted here):

- `sender` "invoked the swap entrypoint and pushed `amountIn` of `tokenIn` beforehand";
- `amountIn` is "the exact amount of `tokenIn` the caller transferred to the propAMM before calling `swap` (push payment); nothing is pulled".

The upstream interface should take the same correction.

0xklapouchy

Fixed in PR#10

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.